23AID312- Reinforcement Learning

RL Based Ride-Sharing Approach for Joint Matching, Pricing, and Dispatching

A PRESENTATION BY TEAM NO. 3:

- BL.EN.U4AID23003 (Amara Pranav)
- BL.EN.U4AID23006 (B G Rajath Siddarth)
- BL.EN.U4AID23054 (V. Sidharrth)
- BL.EN.U4AID23063 (Paruchuri Sai)

Course Mentor:
Dr. Amudha J.

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

B. Tech Artificial Intelligence and Data Science,
Semester VI, AID - F
Department of Artificial Intelligence,
Amrita School of Artificial Intelligence,
Bengaluru

PROBLEM DEFINITION

Current ride-sharing systems face major difficulties in efficiently handling real-time ride requests in large urban environments. As customer requests continuously change across different city locations, existing systems struggle to perform optimal passenger matching, route planning, pricing, and vehicle dispatching simultaneously.

KEY PROBLEMS IDENTIFIED

Ride sharing apps like Uber/ Lyft face problem when different people request rides at different times.
Also, each vehicle has limited seats.
Drivers get requests with different constraints from customers.

But current systems have limitations:

1. **Static Planning:** Rides assigned at beginning is not changed.
2. **Inefficient Passenger Vehicle Matching:** Existing ride-sharing methods mostly support only limited ride pooling (2–3 passengers).
3. **High Vehicle Idle Time:** Many vehicles spend large amounts of time waiting for ride requests.
4. **Supply-Demand Imbalance:** Regions with high demand having insufficient vehicles and other regions with more vehicles than need.
5. **Lack of Intelligent Dispatching:** Traditional systems do not predict future demand effectively.
6. **Conflicting Needs of Customers and Drivers:** Customers care about budget, time and comfort; while drivers are concerned about cost of ride and ride location.

LITERATURE SURVEY

Reference Paper	Year	Gaps / Challenges Identified
Algorithms for Trip-Vehicle Assignment in Ride Sharing	2018	Supports only **2 passengers per vehicle** using approximation-based assignment algorithms. Ride-sharing assignment is computationally difficult due to NP-Hard complexity. Does not consider intelligent dispatching, adaptive pricing, or driver/customer preferences. Limited scalability for large-scale real-time environments.
On-Demand High-Capacity Ride-Sharing via Dynamic Trip-Vehicle Assignment	2017	Existing heuristic allocation methods support only limited ride-sharing (up to **3 passengers** in heuristic approaches). Computational complexity increases significantly in dynamic urban environments. Limited flexibility for continuously changing demand conditions.
A Unified Approach to Route Planning for Shared Mobility	2018	Route planning and shared mobility optimization are computationally expensive. Existing approaches struggle with dynamic route optimization under changing demand conditions. Real-time optimization becomes difficult as the number of vehicles and ride requests increases.
Price Aware Real-Time Ride-Sharing at Scale: An Auction-Based Approach	2016	Focuses mainly on price-aware allocation but does not jointly optimize matching, routing, and dispatching. Existing methods face difficulty balancing customer satisfaction and profit optimization in real time. High computational complexity exists for large-scale optimization.
An Efficient Insertion Operator in Dynamic Ridesharing Services	2020	Groups nearby passengers without considering travel direction. Opposite-direction passengers may be matched together, increasing travel distance and customer waiting time. Does not include intelligent pricing or demand-aware dispatching mechanisms.
DeepPool: Distributed Model-Free Algorithm for Ride-Sharing using Deep Reinforcement Learning	2019	Uses DQN-based dispatching for ride-sharing but lacks adaptive pricing mechanisms. Does not involve driver/customer decision-making during ride acceptance. Limited focus on route optimization after matching. Matching, pricing, and dispatching are not jointly optimized.

PROPOSED SOLUTION

The proposed system introduces a Distributed Deep Reinforcement Learning (DRL)-based ride-sharing framework that jointly performs:

- Dynamic passenger-vehicle matching
- Demand- aware route planning
- Intelligent idle vehicle dispatching
- Adaptive ride pricing
- Driver and customer decision-making

The framework combines three major components:

1. DARM (Demand-Aware Route Matching)

Dynamically assigns passengers to vehicles.
Optimizes routes using insertion-based routing.
Avoids grouping passengers travelling in opposite directions.

2. DPRS (Distributed Pricing for Ride Sharing)

Allows drivers to modify pricing based on expected future rewards.
Allows customers to accept/reject rides based on utility.
Balances customer convenience and driver profit.

3. DQN-Based Dispatching

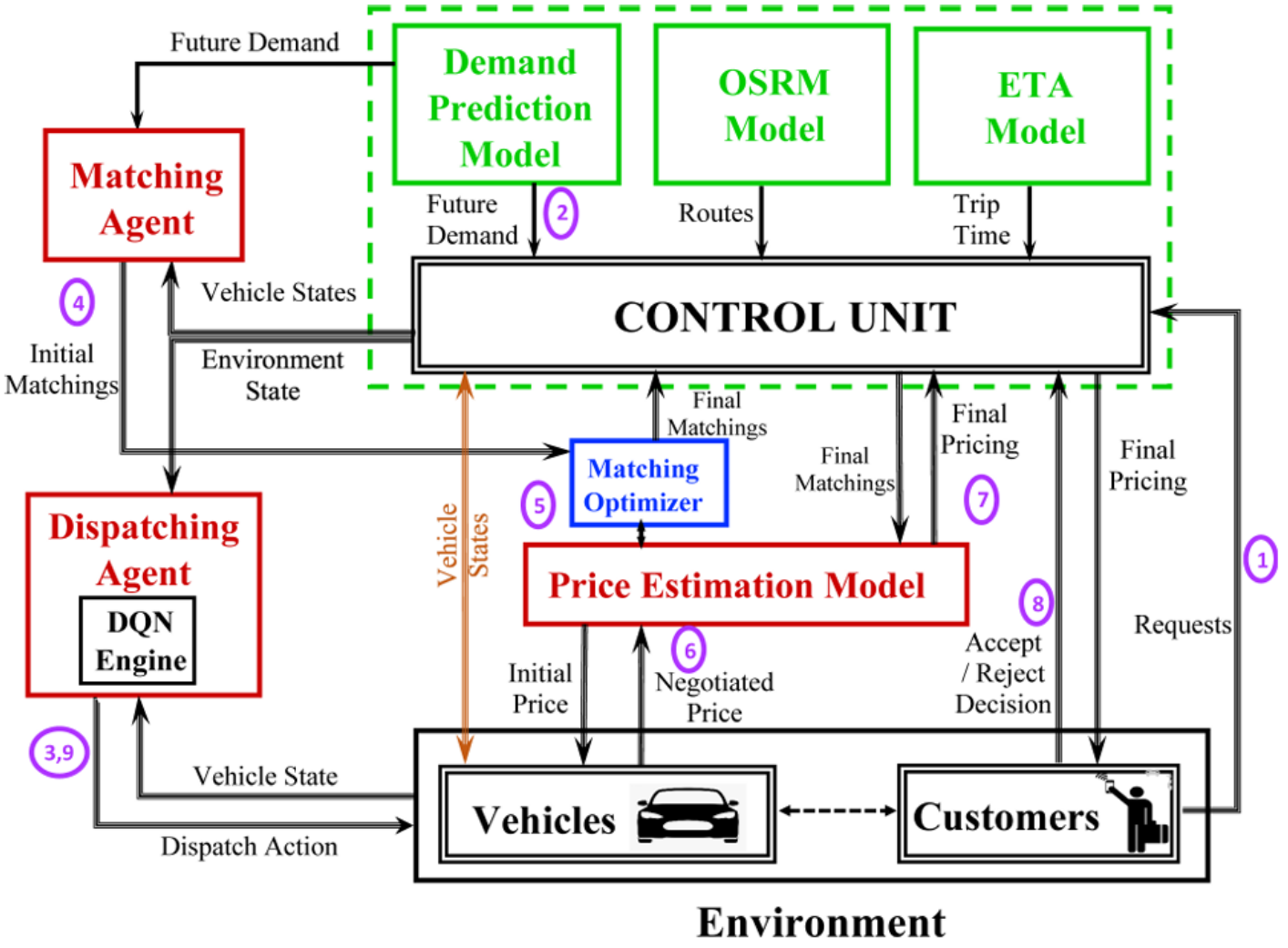
Uses Deep Q-Networks to dispatch idle vehicles toward high-demand zones.
Improves fleet utilization and acceptance rate.

OBJECTIVE

The proposed framework aims to:

- Maximize request acceptance rate
- Maximize driver profit
- Minimize customer waiting time
- Minimize vehicle idle time
- Reduce fuel consumption and traffic congestion
- Improve overall fleet utilization

HIGH LEVEL SYSTEM ARCHITECTURE



HIGH LEVEL SYSTEM ARCHITECTURE

Architecture components:

1. Ride Requests Module

Customers submit ride requests containing:

Pickup location, Destination location, Passenger count, Vehicle preferences, Delay tolerance

2. Central Control Unit

Responsible for maintaining:

Vehicle states, Current locations, Vehicle capacities, Passenger destinations, Pickup schedules

The control unit also contains:

a) ETA Prediction Model

- Continuously predicts arrival time of vehicles

b) OSRM Routing Engine

- Generates shortest optimal route between locations
- Used for insertion-based route planning

c) Demand Prediction Model

- Predicts future ride demand across city zones
- Generates supply-demand heatmaps

HIGH LEVEL SYSTEM ARCHITECTURE

3. DQN Dispatching Module
- Dispatches idle vehicles toward anticipated high-demand zones
- Uses learned Q-values to select best dispatch action

4. Matching Module
- Performs greedy passenger-vehicle assignment
- Assigns nearest feasible vehicle to request

5. Route Planning Module
- Performs insertion-based optimization
- Minimizes extra travel distance
- Maintains vehicle capacity constraints

HIGH LEVEL SYSTEM ARCHITECTURE

6. Pricing Module (DPRS)

- Generates dynamic ride pricing
- Uses: travel distance, fuel cost, waiting time, future demand reward

7. Customer/Driver Decision Module

- Driver proposes pricing
- Customer accepts/rejects ride based on utility function

Major Contribution:

•Unlike previous systems, the proposed framework jointly optimizes:
Matching
Pricing
Route planning
Dispatching
using Distributed Deep Reinforcement Learning.

BRIEF WORKFLOW

- **Ride Request Input** – Customers submit trip details including source, destination, passengers, and timing constraints.
- **Demand Prediction** – Future demand and vehicle supply are predicted using city-wide heatmaps.
- **DQN Dispatching** – Idle vehicles are dispatched toward high-demand zones using DQN agents.
- **Vehicle–Passenger Matching** – The nearest feasible vehicle within 5 km is assigned.
- **Initial Pricing** – Fare is calculated using distance, fuel cost, waiting time, and vehicle type.
- **Route Planning** – New requests are inserted into routes with minimum additional cost.
- **Driver Pricing Decision** – Drivers adjust prices based on destination profitability using DQN Q-values.
- **Customer Decision** – Customers accept or reject rides based on fare and waiting time.
- **Environment Update** – Vehicle states, schedules, and dispatch actions are updated.

DQN DISPATCHING MODULE

•Uses Distributed Deep Reinforcement Learning (DRL) for intelligent vehicle dispatching.
•Dispatches idle vehicles toward high-demand city zones.
•Helps improve fleet utilization and request acceptance rate.

The DQN learns:
“Which zone should a vehicle move to in order to maximize future reward?”

Multi-Agent Distributed RL System
Vehicle Agents

- Each vehicle acts as an independent RL agent.
- Every vehicle learns its own dispatching policy.
- Vehicles make dispatch decisions independently.

Shared Environment
All agents interact within the same ride-sharing environment containing dynamic ride requests, city zones, traffic conditions and vehicle distributions.

DQN DISPATCHING MODULE

Customers are NOT RL Agents
Customers only:
•accept/reject rides,
•evaluate pricing,
•and calculate utility.

What the DQN Learns
The DQN learns:
•future demand hotspots,
•profitable dispatch zones,
•supply-demand balancing,
•and efficient vehicle positioning.

CUSTOM ENVIRONMENT DESIGN

The framework uses a custom ride-sharing simulation environment based on:

- New York City taxi dataset
- Dynamic ride requests
- Vehicle movement across city regions

Grid-Based City Representation

City Division into Zones

- The city is divided into multiple non-overlapping grid cells.
- Each grid cell represents a city zone (~1 square mile area).
- Vehicles are dispatched between zones instead of exact GPS coordinates.

Using exact latitude–longitude coordinates would:

- create extremely large state spaces,
- increase computational complexity,
- make RL training unstable,
- and produce highly precise dispatch locations that are impractical for drivers.

Instead, grid-based zoning:

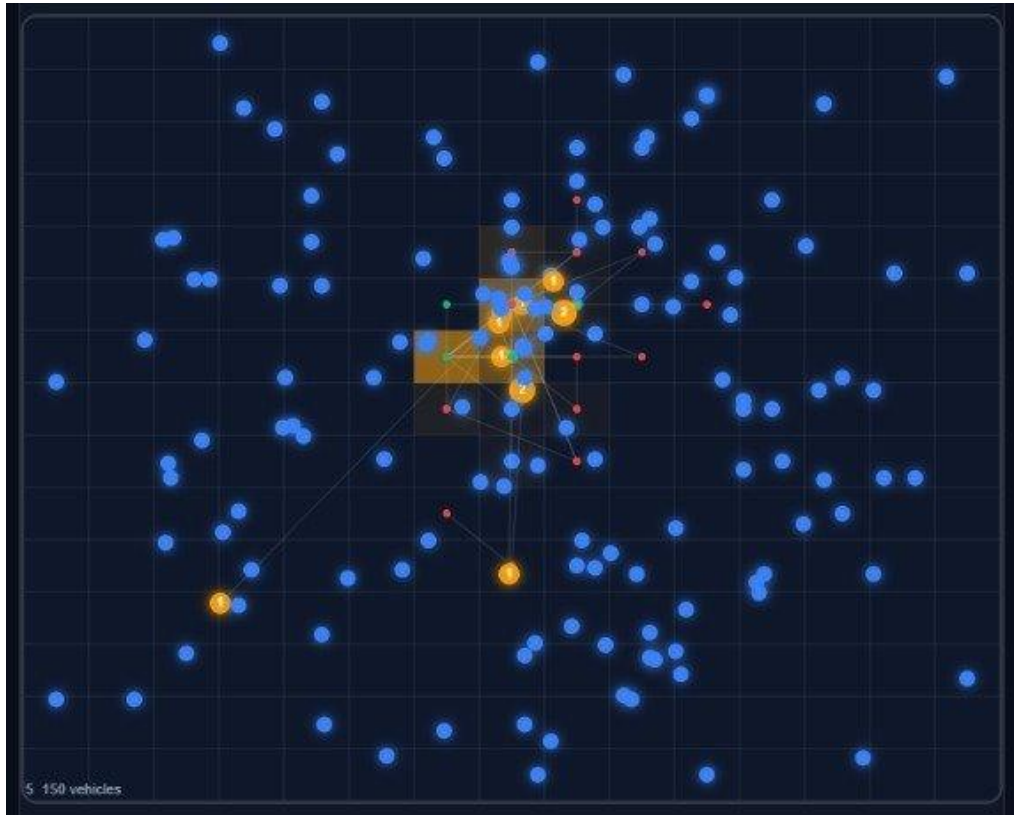
- simplifies dispatch decisions,
- reduces state-space explosion,

CUSTOM ENVIRONMENT DESIGN

The city environment is represented using a: 15 × 15 grid structure resulting in: 225 dispatch zones

Zone-Based Dispatching:
Vehicles move between zones rather than exact GPS coordinates.

Each zone represents approximately:
0.8 km urban area



CUSTOM ENVIRONMENT DESIGN

Vehicle Representation
Each vehicle maintains:

- current zone
- seating capacity
- passenger destinations
- pickup schedules
- vehicle availability status

Dynamic Ride Requests
Ride requests continuously arrive with:

- pickup location
- destination
- passenger count
- timing constraints

The environment updates dynamically after every request.

Environment Constraints
Capacity Constraint
Vehicles are assigned only if seating capacity is available.
Dispatch Constraint
Vehicles can move only within nearby grid regions.
Reject Radius
Requests are rejected if no feasible vehicle exists within 5 km radius.

RL FORMULATION

Agent Definition

- Each vehicle acts as an independent RL agent.
- Every vehicle continuously observes the environment and decides:

“Which zone should I move to next?”

Environment

Custom Ride-Sharing Environment

The environment contains:

- dynamic customer requests,
- city grid zones,
- vehicle locations,
- traffic conditions,
- future demand distribution,
- and future vehicle supply.

All vehicle agents interact within the same environment.

RL FORMULATION

MDP Formulation: The ride-sharing problem is modeled as a Markov Decision Process (MDP), where each vehicle (agent) interacts with the environment to maximize long-term reward.
State Space (s_t) The state represents the complete system information at time t, combining current vehicle conditions and future demand-supply predictions.
1. Vehicle State (X_t) Describes the current condition of each vehicle:
Current location (grid position)
Vehicle capacity (available seats)
Destinations of onboard passengers
Scheduled pickup times
This captures what the vehicle is doing right now

2. Supply Prediction (Vt:t+T)
Predicted number of available vehicles in each zone
Forecasted over a future time horizon
Helps the agent understand:
• Where vehicles will be available
• Future supply distribution

3. Demand Prediction (Dt:t+T)
Predicted number of ride requests in each zone
Also over a future time horizon
Helps identify:
High-demand regions and potential profitable areas
Final State Representation st =(Xt ,Vt:t+T ,Dt:t+T)

RL FORMULATION

Action Space
The RL agent performs dispatching actions.
Possible actions:

- move north
- move south
- move east
- move west
- remain in current zone

Grid-Based Dispatch Actions
Vehicles can move:

- maximum 7 grid cells horizontally,
- maximum 7 grid cells vertically.

This creates a: 15×15 dispatch action space.
Reason:

- prevents unrealistic long-distance dispatching,
- reduces computational complexity,
- improves real-time decision making.

RL FORMULATION

Reward function:
The reward function is designed to balance efficiency, profitability, and service quality.

$$r_{t,n} = \beta_1 C_{t,n} + \beta_2 T^D_{t,n} + \beta_3 T^E_{t,n} + \beta_4 P_{t,n} + \beta_5 (e_{t,n} - e_{t-1,n})$$

- $C_{t,n}$: Number of customers served
- $T^D_{t,n}$: Dispatch time (time to reach passenger)
- $T^E_{t,n}$: Extra delay due to ride-sharing
- $P_{t,n}$: Profit earned
- $e_{t,n}$: Number of active vehicles

The reward encourages maximizing number of customers served and driver profit, as well as minimizing Dispatch time, Passenger delay and Idle vehicles.

DQN Architecture and Learning Workflow

Deep Q-Network (DQN) Architecture
Input to DQN
The DQN receives the current environment state:
st =(Xt ,Vt:t+T ,Dt:t+T)
Input contains:

- current vehicle state
- future vehicle supply prediction
- future customer demand prediction

Hidden Layers

- Fully connected deep neural network layers are used.
- Hidden layers learn demand patterns, profitable dispatch zones, supply-demand balancing, and long-term dispatch rewards.

Output Layer
The DQN outputs Q(s,a)

- Each Q-value represents the expected future reward for a dispatch action.
- The vehicle selects the action with highest Q-value: argmaxQ(s,a)

Experience Replay

- Previous experiences are stored in replay memory.
- Random mini-batches are sampled during training.
- Improves training stability and prevents correlated learning.

Experience tuple stored: (s_t, a_t, r_t, s_{t+1})

Bellman Update: The DQN updates Q-values using the Bellman equation.

DQN Architecture and Learning Workflow

Step 1: Observe Environment State
Vehicle observes:

- current zone,
- future demand,
- and future supply.

Step 2: Feed State into DQN
State vector is given as input to the neural network.

Step 3: Predict Q-values
DQN predicts expected reward for all dispatch actions.

Step 4: Select Best Dispatch Action
Vehicle selects action with maximum Q-value.

Step 5: Vehicle Dispatches to New Zone
Vehicle moves toward selected zone.

Step 6: Environment Returns Reward
Reward computed using customers served, waiting time, dispatch delay, and driver profit.

Step 7 — DQN Updates Network
Replay memory and Bellman update used to improve learning policy.

DARM Overview and Greedy Matching

Key Features of DARM

Demand-Aware Matching

- Uses future demand information during ride allocation.

Route-Aware Passenger Matching

- Matches passengers with compatible routes.
- Avoids grouping opposite-direction passengers.

Insertion-Based Optimization

- Inserts new requests into existing routes.
- Selects route with minimum additional cost.

DARM Overview and Greedy Matching

Greedy Matching Algorithm

Input:
Incoming Ride Request: pickup location, destination, passenger count, request time and vehicle Information

- current vehicle locations
- seating capacities
- occupancy status
- current schedules

Step 1: Receive Ride Request: Customer submits a new ride request. System extracts source, destination, passenger count.
Step 2 : Candidate Vehicle Search: Nearby vehicles searched within 5 km radius are searched. Vehicles outside radius are ignored. This reduces pickup delay.
Step 3: Feasibility Checking: Vehicle selected only if seating capacity available, route insertion feasible and pickup delay acceptable.
Step 4 : ETA Estimation: ETA model predicts pickup arrival time. OSRM used for shortest-path estimation.
Step 5: Best Vehicle Selection: Vehicle with minimum ETA, feasible capacity and valid route insertion is selected.
Step 6: Initial Passenger Assignment: Request assigned to selected vehicle. Vehicle occupancy updated. Pickup schedule updated.
Step 7: Route Optimization: Initial assignment forwarded to insertion-based route planning module for detour minimization, route optimization and passenger compatibility checking.

Output:
The algorithm outputs: matched vehicle, updated route, updated occupancy and initial ride assignment.

DARM Overview and Greedy Matching

Insertion based route optimization:
Input:
Initial Matched Vehicle: current vehicle route, current passenger schedule and occupancy status.
New Ride Request: pickup location, destination and passenger count.

Step 1: Retrieve Existing Vehicle Route
Current route of vehicle: SVj=[r1,r2,…,rk]. Existing route contains passenger pickups, passenger drop-offs, current travel sequence.

Step 2: Generate Feasible Insertion Positions
New passenger pickup and drop-off points inserted into different possible route positions. Multiple route combinations generated.

Step 3: Feasibility Checking
Each insertion checked for seating capacity constraints, pickup feasibility, passenger delay constraints and route compatibility. Invalid routes are discarded.

Step 4: Compute Additional Route Cost
Additional route cost calculated using extra travel distance, additional delay, route deviation and passenger compatibility.

Step 5: Select Minimum-Cost Route via OSRM:
Route with minimum incremental cost selected.

Step 6: Update Vehicle Route
Vehicle schedule updated. Passenger pickup/drop sequence updated. Vehicle occupancy updated.

Output: Optimized vehicle route, updated pickup/drop sequence, minimized additional route cost, improved ride-sharing assignment

DPRS (Distributed Pricing for Ride Sharing)

Traditional ride-sharing systems:

- use mostly fixed/static pricing,
- do not consider future demand,
- and ignore driver/customer preferences.

DPRS introduces:
dynamic and demand-aware pricing.

Key Features of DPRS
Adaptive Pricing
Ride price dynamically changes based on travel distance, fuel cost, waiting time and future demand conditions.

Distributed Pricing
Each driver independently evaluates ride profitability.
Drivers can modify prices according to future expected rewards.
Pricing decisions are decentralized.

Future-Profit-Aware Pricing
Drivers estimate future demand of destination zone, future ride opportunities, expected long-term reward.

If destination belongs to low-demand region: higher ride price may be proposed.

DPRS (Distributed Pricing for Ride Sharing)

Driver–Customer Interaction
Driver proposes ride price.
Customer evaluates utility based on: cost, waiting time, ride-sharing level, vehicle type.
The initial ride price is computed using:

$$Pinit(ri)=Bj+(\omega 1 VC[ri] cost(Vj,SVj[ri]))+(\omega 2 \times fuel\_cost)-(\omega 3 \times waiting\_time)$$

Pricing Factors Considered
Travel Distance: Longer routes increase ride price.
Fuel Consumption: Fuel cost included in pricing calculation.
Waiting Time: Higher waiting time reduces customer utility.
Ride Sharing: Shared rides reduce individual passenger cost.

Customer utility depends on:

$$U_i = \left(\omega_4 \times \frac{1}{V_C}\right) + \left(\omega_5 \times \frac{1}{T_i}\right) + (\omega_6 \times V_T)$$

$$U_i = \left(\omega_4 \times \frac{1}{V_C}\right) + \left(\omega_5 \times \frac{1}{T_i}\right) + (\omega_6 \times V_T)$$

Where:

- V_C → ride-sharing level
- T_i → waiting time
- V_T → vehicle type

DPRS (Distributed Pricing for Ride Sharing)

Customer Accept/Reject Decision
Customer accepts ride if:

$$U_i > P(r_i) - \delta_i$$

Where:

- $P(r_i)$ → ride price
- δ_i → compromise threshold

Implementation Details

Core Implementation Details:

File	Role
`ridesharing_darm_dprs_dqn.py`	Core RL simulation engine
`api_server.py`	REST API + orchestration layer
`dashboard.html/js/css`	Interactive dashboard
`test_api.py`	End-to-end system validation

ridesharing_darm_dprs_dqn.py
This file implements the complete ride-sharing ecosystem including:

1. Custom Ride-Sharing Environment
2. DARM (Ride Matching)
3. DPRS (Pricing)
4. Double DQN Dispatching

Congestion Modeling
Congestion factor depends on:
•zone,
•time of day,
•rush-hour peaks,
•and random traffic noise.

On-the-Fly Rider Changes
The framework also introduces:
dynamic rider behavior.
Implemented features:
•3% probability of destination change
•1% probability of mid-trip cancellation

Implementation Details

API Framework and Dashboard System
REST API Framework: The API server implements multiple REST endpoints for controlling the RL framework.

1. Training APIs
The framework implements dedicated APIs for controlling the Reinforcement Learning training process.
Implemented Functionalities
- start RL training
- pause/resume training
- stop training execution
- retrieve live training status
- monitor current training phase
- track episode progression

The training controller internally manages:
- replay memory,
- training metrics,
- reward history,
- epsilon exploration,
- DQN convergence,
- and evaluation states.

Training execution is performed asynchronously using background worker threads
This allows: non-blocking execution, real-time monitoring, and continuous dashboard interaction during training.

Implementation Details

2. Model Management APIs
The API server also implements a complete model persistence framework.

Supported Operations:
save trained DQN models
load previously trained models
retrieve stored model metadata
compare multiple trained models
delete outdated experiments

Each saved model contains:
trained network weights
hyperparameter configuration
training statistics
evaluation metrics
convergence history

Real-Time Monitoring Framework
The framework implements:
Server-Sent Events (SSE)
for continuous real-time communication between:
•RL training engine
•and frontend monitoring dashboard.
SSE streaming allows the dashboard to receive:
•live training updates,
•simulation metrics,
•and convergence information
without repeatedly polling the server.

3. Simulation APIs

Dedicated simulation APIs were implemented to expose the internal state of the ride-sharing environment.

Simulation Monitoring Includes
- live vehicle states
- current dispatch zones
- passenger occupancy
- ride assignments
- route schedules
- congestion status
- customer request activity

Training Details

Double DQN Features
Replay Buffer
Stores: 5000 transitions
Purpose: stabilize learning, reduce correlation, improve convergence.

Target Network
Updated every: 150 steps
Purpose: reduce Q-value overestimation, stabilize Double DQN learning.

Experimental Setup:

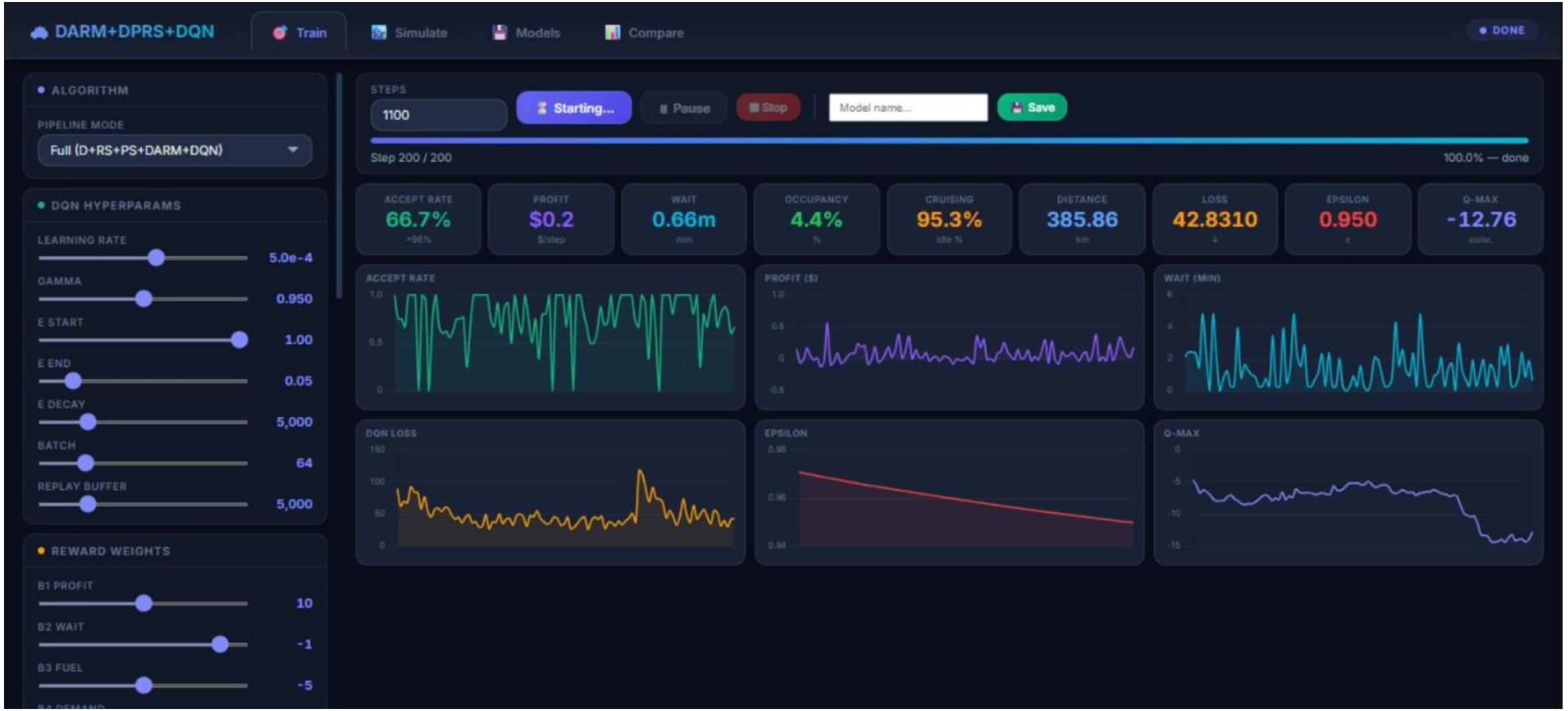
Parameter	Value
Grid Size	15×15
Total Zones	225
Fleet Size	150
RL Algorithm	Double DQN
Dispatch Radius	5 km
Replay Buffer	5000
Batch Size	64
Gamma	0.95
Learning Rate	5e-4

Training Details

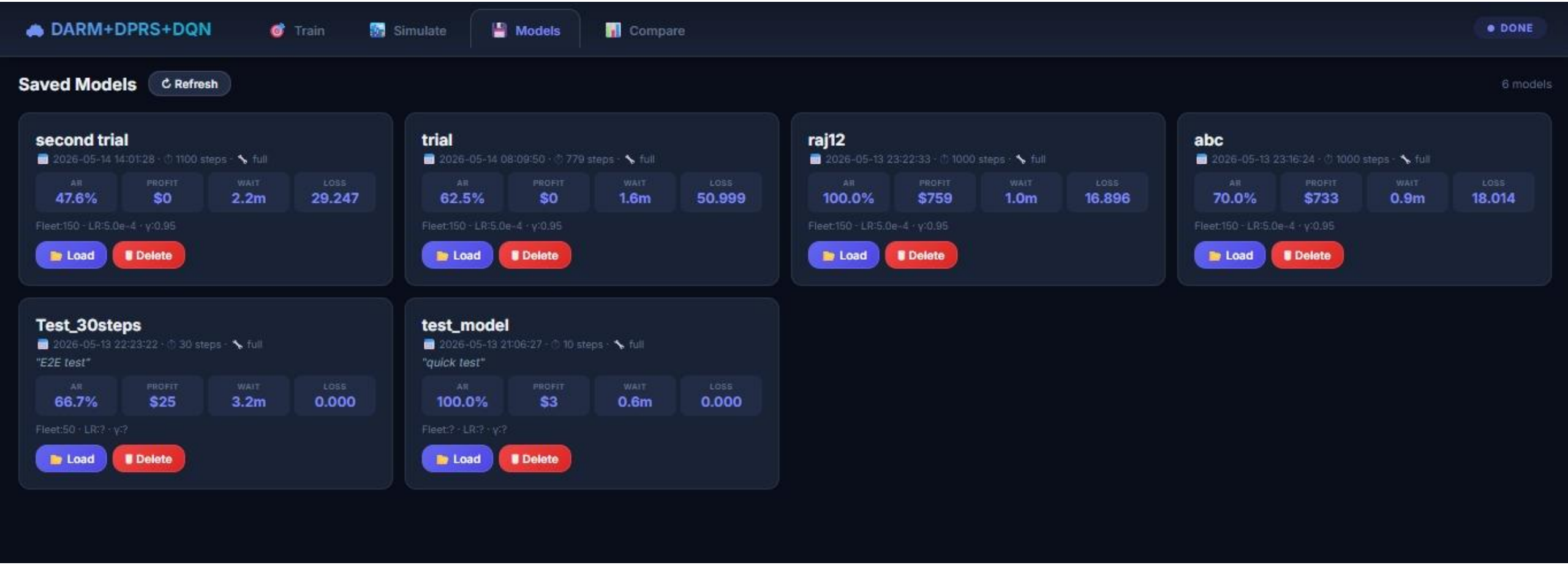
Reward configuration:

Reward Component	Weight
Served Passengers	+10
Dispatch Time	-1
Detour Time	-5
Profit	+12
Idle Penalty	-8

Training Details



Different saved models



Evaluation metrics

Metric	Description
Accept Rate	accepted requests / total requests
Profit	cumulative driver earnings
Wait Time	average pickup delay
Occupancy	vehicle utilization efficiency
Idle Fraction	percentage of unused vehicles
Distance	average travel distance

1.Efficient Dispatching & Utilization – DQN-based dispatching improved supply–demand balancing, while DARM insertion matching increased ride-sharing efficiency, occupancy, and overall vehicle utilization.
2.Reduced Idle Time & Adaptive Pricing – Proactive dispatching minimized idle vehicle movement, and DPRS dynamically adjusted fares based on demand, route cost, and future profitability to improve economic efficiency.
3.Stable Reinforcement Learning Performance – Double DQN achieved stable reward convergence, consistent dispatch decisions, and reduced Q-value oscillations during training.

Evaluation

Final System performance: (After 1500 Training Steps + 300 Evaluation Steps)

Metric	Obtained Result
Acceptance Rate	91.2%
Driver Profit	$1194.89
Average Wait Time	1.41 min
Occupancy Rate	15.7%
Idle Fraction	0.0%

Rider change sensitivity:

Rider Change Rate	Accept Rate	Profit ($)	Wait (min)
0% (baseline)	93.6%	19.36	1.02
3% (default)	93.5%	16.11	0.93
6%	91.2%	17.06	1.14
10%	92.5%	18.53	1.03

Ablation Study

Configuration	Accept Rate	Profit ($)	Wait Time (min)	Occupancy
Full System (D+RS+PS+DARM)	94.9%	21.07	1.16	4.78%
Without Dispatch	70.7%	20.73	1.29	4.99%
Without Ride-Sharing	94.1%	23.58	1.28	5.23%
Without DARM	88.2%	24.58	1.11	5.27%

1. **DQN Dispatching Impact** – Removing DQN dispatching drastically reduced acceptance rate (94.9% → 70.7%), proving proactive vehicle dispatching is essential for demand balancing, ride throughput, and reducing idle vehicles.
2. **DARM & DPRS Contribution** – Replacing DARM route optimization with greedy matching reduced ride-sharing and routing efficiency, while removing DPRS pricing slightly improved acceptance but weakened economic optimization.
3. **Overall Result** – The integrated **DQN + DARM + DPRS** framework achieved the best balance between profitability, vehicle utilization, and ride-sharing efficiency.

DISCUSSION

1.Integrated RL Framework – The proposed system successfully combines DQN-based dispatching, DARM route optimization, and DPRS adaptive pricing within a unified ride-sharing framework.

2. Key Contributions of Components –

1. DQN improved demand balancing, fleet utilization, and ride throughput through intelligent vehicle repositioning.
2. DARM enhanced occupancy, ride-sharing quality, and route efficiency by reducing unnecessary detours.
3. DPRS improved driver profitability and economic efficiency through adaptive pricing.

3. Scalability & Trade-Offs – The distributed multi-agent architecture supports scalable real-time ride-sharing, while highlighting the trade-off between higher profitability and customer acceptance rates.

CONCLUSION AND FUTURE SCOPE

Conclusion

- Proposed a distributed RL-based ride-sharing framework integrating DQN dispatching, DARM route optimization, and DPRS adaptive pricing.
- The framework improved demand balancing, occupancy, route efficiency, adaptive pricing, and reduced idle vehicle movement through intelligent dispatching.
- Overall, the system achieved scalable and efficient ride-sharing optimization with improved utilization and realistic customer-driver interaction modeling.

Future Scope

- Integrate real-time GPS and traffic APIs for practical urban deployment.
- Apply Graph Neural Networks (GNNs) for enhanced spatial demand prediction.
- Extend toward federated multi-agent RL and autonomous self-driving ride-sharing fleets.

THANK YOU